

1. Four Ways to Create a Thread

Extend Thread class: Override the run() method. It is simple but limits inheritance as Java does not support multiple inheritance.

Implement Runnable interface: Implement run() and pass it to a Thread object. This separates the task from the thread, promoting better design.

Implement Callable interface: Implement call() method. Unlike Runnable, it supports returning a result and throwing checked exceptions, typically used with FutureTask.

Using Thread Pools (ExecutorService): The recommended way in production. It reuses existing threads, which significantly reduces the overhead of frequent thread creation and destruction.

2. Thread Lifecycle and Transitions

The thread lifecycle consists of 6 states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED.

Transitions: A thread enters RUNNABLE after start(). It moves to BLOCKED when waiting for a monitor lock. It enters WAITING by calling wait() or join(), and TIMED_WAITING via sleep() or wait(timeout). Once the task completes, it reaches TERMINATED.

3. How Thread Pools Work

When a task is submitted:

If current pool size < core pool size, a new thread is created.

If core threads are full, the task is added to the work queue.

If the queue is full and current pool size < max pool size, a new temporary thread is created.

If max pool size is reached and queue is full, the Rejection Policy is triggered.

4. Risks of newCachedThreadPool and newFixedThreadPool

newCachedThreadPool: Its max thread count is Integer.MAX_VALUE. High concurrency can spawn too many threads, leading to OOM (Out of Memory).

newFixedThreadPool: Its work queue is an unbounded LinkedBlockingQueue. If tasks accumulate faster than they are processed, the queue will grow indefinitely, also causing OOM.

5. Future vs. CompletableFuture

Future: Introduced in Java 5. It represents an asynchronous result, but it is "blocking"—you must call get() to wait for the result, which kills concurrency.

CompletableFuture: Introduced in Java 8. It allows for non-blocking callback chains, combining

multiple tasks, and handling exceptions asynchronously. It is significantly more powerful for modern, reactive programming.

6. Lock vs. synchronized

synchronized: A JVM-level keyword. It is automatic (locking/unlocking), but inflexible (cannot be interrupted, cannot try-lock, and is not fair).

Lock (e.g., ReentrantLock): A Java API. It provides advanced features like fairness, the ability to interrupt a thread waiting for a lock, and the ability to attempt to acquire a lock without blocking indefinitely.

7. Thread Communication Methods

wait(): Causes the current thread to release the lock and enter a waiting state.

notify() / notifyAll(): Wakes up a single thread or all threads waiting on the object's monitor.

join(): The current thread waits until the thread it joins finishes its execution.

8. Common CompletableFuture APIs

runAsync / supplyAsync: Used to trigger asynchronous tasks. runAsync takes a Runnable (no return), supplyAsync takes a Supplier (returns a result).

thenApply / thenApplyAsync: Used to process/transform the result of a previous stage once it completes.

handle / exceptionally: Used to manage errors. handle allows processing both results and exceptions; exceptionally specifically handles exceptions.

thenCompose / thenCombine: thenCompose flattens nested futures (connecting dependent tasks); thenCombine merges results from two independent futures.

allOf / anyOf: allOf waits for all futures to complete; anyOf completes as soon as any one of the provided futures completes.